



## **Session 06:**

# **Modern JavaScript (ES6) Features**



# NỘI DUNG

1. Hiểu Template String và sử dụng nội suy chuỗi
2. Sử dụng Destructuring để trích xuất Array và Object
3. Áp dụng toán tử ba chấm vào Spread và Rest
4. Hiểu bất đồng bộ và vận dụng Promise, Async/Await
5. Biết cách chia nhỏ và sử dụng Module hiệu quả

# 1. Module

Chia nhỏ mã nguồn, xây dựng cấu trúc mã nguồn riêng biệt, dễ bảo trì, dễ tái sử dụng



# Cú pháp Export

## Named Export

```
// Named Export
export const PI = 3.14159;
export function add(a, b) {
    return a + b;
};
export function subtract(a, b) {
    return a - b;
};
```

Xuất nhiều biến/hàm. Khi import bắt buộc phải dùng đúng tên và có ngoặc nhọn { }

## Default Import

```
// Default Export
export default function multiply(a, b) {
    return a * b;
}
```

Mỗi file chỉ có **Một** export default. Khi import có thể tự do đặt tên, không dùng ngoặc nhọn

# Cú pháp Import

## Named Import

```
import { PI, add } from "./math.js";  
console.log(PI); // 3.14159  
console.log(add(2, 3)); // 5
```

## Đổi tên / Alias

```
import { add as sum } from "./math.js";  
console.log(sum(2, 3)); //5
```

## Default Import

```
import initializeApp from "./app.js";
```

Không có { }, tên biến tùy chọn do dev tự đặt

## Gộp tất cả

```
import * as math from "./math.js";  
console.log(math.PI); //3.14159  
console.log(math.add(2, 3)); // 5
```

HTML Integration

```
<script type="module" src="app.js"></script>
```

Bắt buộc khai báo type='module' để trình duyệt hiểu đây là hệ thống module ES6.

## 2. Rest Parameters

Thay thế hoàn toàn object “**arguments**” cũ trong Arrow Functions

```
// Arrow Function with Rest
const arrowFunctionWithRest = (...args) => {
  console.log(args);
};
arrowFunctionWithRest(1, 2, 3);
// output: [1, 2, 3]
```

```
// Practical Use Case (Sum with .reduce())
const sum = (...numbers) => {
  return numbers.reduce((total, num) =>
    total + num, 0);
};
console.log(sum(1, 2, 3, 4)); // 10
console.log(sum(5, 10, 15)); // 30
```

**Quy tắc vàng:** Tham số Rest phải luôn đặt ở vị trí cuối cùng trong danh sách tham số

## 3. Spread Operator

Sao chép

```
const copyListNumber = [...listNumber];  
console.log(copyListNumber); // output: [1, 2, 3, 4]
```

**Lưu ý:** Chỉ là **Shallow Copy** (Sao chép nông).  
Tham chiếu gốc bên trong object/mảng lồng  
nhau vẫn bị ảnh hưởng!

Gộp mảng (Merging)

```
const listNumber = [1, 2, 3, 4];  
const listName = ["Nam", "Linh"];  
const mergeArray = [...listNumber, ...listName];  
console.log(mergeArray); //output: [1, 2, 3, 4, "Nam", "Linh"]
```

Object Spread

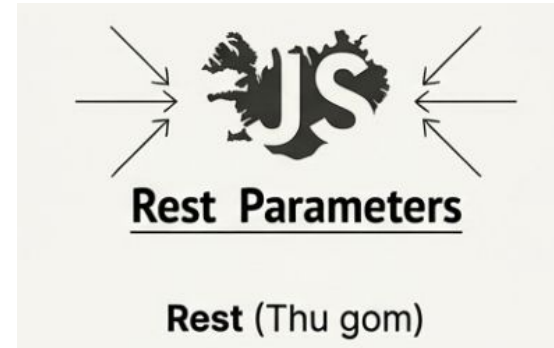
```
const infoUser = { id: 1, age: 22 };  
const detailUser = { ...infoUser, age: 20, name: "Dev" };  
console.log(detailUser); //output; {id: 1, age: 20, name: "Dev"}  
console.log(infoUser); //output; {id: 1, age: 22} // Tuổi không bị thay đổi
```

Giữ nguyên **infoUser**, ghi đè **age** thành 20

# So sánh Spread & Rest



- Mở rộng/ phá vỡ cấu trúc của mảng hoặc đối tượng.
- **Ngữ cảnh:** Dùng khi thực thi hoặc Gán
- **Ví dụ:** Tạo mảng mới, gộp object, truyền tham số cho hàm



- Gom nhiều giá trị riêng lẻ lại thành 1 mảng duy nhất
- **Ngữ cảnh:** Dùng khi khai báo (Declaration)
- **Ví dụ:** Nhận tham số không giới hạn trong Function, hoặc phần dư trong Destructuring



## 4. Template String

Quá khứ

```
let greeting = "Hello" + name  
let greeting = "Hello" + name +  
  , "welcome to" + system + "!";
```

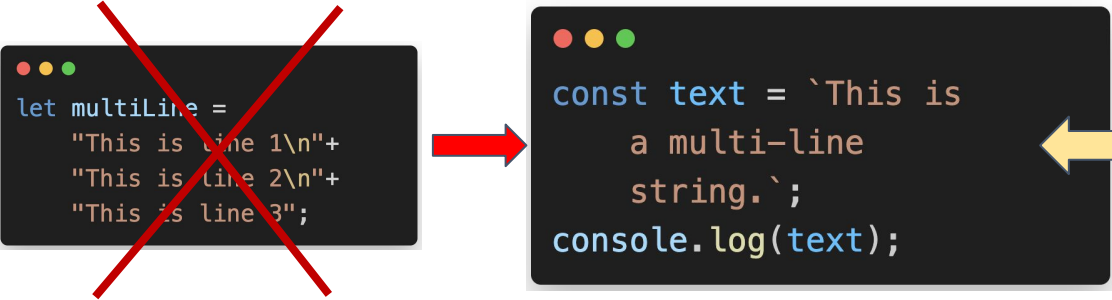
Dễ nhầm lẫn khoảng trắng,  
rườm rà, khó đọc

Hiện đại

```
let greeting = `Hello ${name},  
welcome to ${system}!`;
```

Dùng dấu backtick (`). Cú pháp rõ  
ràng, trực quan

# Nội suy (Interpolation) & Đa dòng (Multi-line)



```
let multiLine =  
  "This is line 1\n"+  
  "This is line 2\n"+  
  "This is line 3";
```

```
const text = `This is  
  a multi-line  
  string.`;  
console.log(text);
```

Không cần ký tự \n hay  
toán tử nối chuỗi

```
// Embedded Expressions
```

```
const getGreeting = (name) => `Hello, ${name}!`;  
console.log(`${getGreeting("Alice")}`);
```

Chèn biến, phép toán,  
hoặc gọi hàm trực tiếp  
vào chuỗi

# Object Destructuring

Object Destructuring - “**Bóc tách**” Đối Tượng: Phá vỡ cấu trúc đối tượng để trích xuất dữ liệu nhanh thành cách biến độc lập

```
const infoUser = { id: 1, name: "Dev", age: 22 };
```

## Anatomy of Code

```
let { name } = infoUser;
```

```
console.log(name); // Dev
```

# Object Destructuring

## Đổi tên biến

```
let { name: nameUser } = infoUser;  
console.log(nameUser);  
//Tên biến thành nameUser = "Dev"
```

Trích xuất "name" nhưng  
gán vào biến "nameUser"

## Giá trị mặc định

```
const { name, age = 22 } = obj;
```

Nếu obj không có "age",  
giá trị fallback sẽ là 22

# Array Destructuring

## Trích xuất Mảng

```
Fira Code
let [a, b, c] = [100, 200, 300];
```

Bỏ qua phần tử

Amber Dùng dấu phẩy (,) để bỏ qua vị trí không cần thiết.

```
Fira Code
const [, , number3] = obj;
```

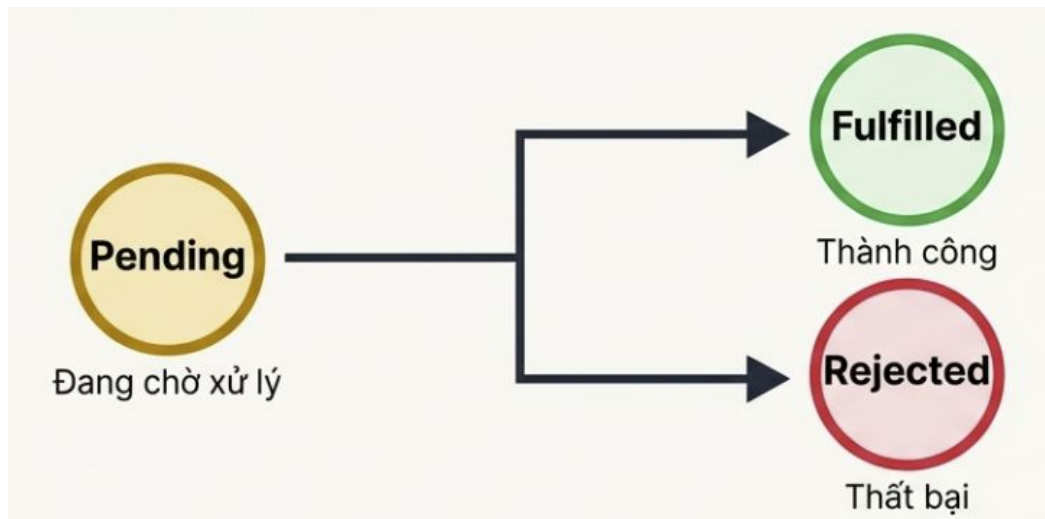
## Tuyệt chiêu Function Params

```
Fira Code
function updateProfile({ name, age, role }) {
  console.log(`Cập nhật: ${name}`);
}
```

**Destructuring** ngay tại tham số hàm. Nhận thẳng Object và bóc tách tức thì, code cực kỳ ngắn gọn.

# Asynchronous

**Tại sao cần Async? Giúp** JavaScript thực hiện các tác vụ tốn thời gian (gọi API, đọc file) mà không làm **“treo”** giao diện (Non-blocking UI)



```
.then(data => {...})
```

```
.catch(error => {...})
```

Promise giải quyết vấn đề Callback Hell nhờ khả năng nối chuỗi (chaining)

# Async/Await

Cú pháp đường dẫn (Syntactic sugar) bọc ngoài Promise. Giúp viết mã bất đồng bộ rành mạch như mã đồng bộ

## Promise `.then()`

```
fetch("api/data")
  .then(res => res.json())
  .then(data => {
    console.log(data);
  })
  .catch(err => console.error(err));
```

## Async / Await

```
async function getData() {
  const res = await fetch("api/data");
  const data = await res.json();
  console.log(data);
}
```

**Tạm dừng** thực thi dòng lệnh cho đến khi Promise giải quyết. Không còn lồng nhau

# Xử lý bất đồng bộ thực tế

Gói các dòng lệnh có thể dùng “await” vào block try

Bắt lỗi cực bộ cực kỳ tốt, thay thế hoàn toàn “.catch” của chuỗi Promise

```
async function fetchUserProfile(userId) {  
  try {  
    const response = await fetch(`/api/users/${userId}`);  
    const user = await response.json();  
    return user;  
  } catch (error) {  
    console.log("Lỗi tải dữ liệu:", error.message);  
  }  
}
```



- ❑ Hiểu và sử dụng được Module để chia nhỏ, tái sử dụng và quản lý mã nguồn hiệu quả thông qua import và export
- ❑ Phân biệt và áp dụng thành thạo Spread Operator để sao chép/gộp dữ liệu và Rest Parameters để xử lý tham số không giới hạn
- ❑ Sử dụng được Template String để tối ưu hóa việc nối chuỗi, viết chuỗi nhiều dòng và nội suy biểu thức một cách trực quan
- ❑ Làm chủ kỹ thuật Destructuring để trích xuất dữ liệu nhanh chóng từ mảng và đối tượng, giúp mã nguồn ngắn gọn và tường minh hơn
- ❑ Hiểu và áp dụng được lập trình bất đồng bộ với Promise và Async/Await để xử lý các tác vụ tốn thời gian mà không gây "treo" giao diện



HỌC VIỆN ĐÀO TẠO LẬP TRÌNH CHẤT LƯỢNG NHẬT BẢN